

VUE.JS

Aula 1: Primeiros Passos

Reatividade sem complicação: do zero ao primeiro componente interativo

O que vamos ver hoje

1

Por que usar um framework front-end?

JS puro x Vue: o problema que o framework resolve

2

O que é o Vue.js

Conceito de reatividade e instância da aplicação

3

Instalando via CDN

Sem Node, sem build — só um `<script>`

4

`data()`, interpolação e diretivas

`{{ }}`, `v-bind`, `v-if` / `v-else`, `v-show`

5

Prática

Contador de cliques + mensagem condicional

AULA 1 / 7

Ao final desta aula você já terá criado sua primeira aplicação Vue reativa, sem precisar instalar nada.

Por que usar um framework?

Vocês já sabem manipular o DOM com JavaScript puro. O problema aparece quando a tela cresce.

JavaScript puro

- ✗ `querySelector` / `getElementById` em todo lugar
- ✗ Você mesmo sincroniza dados ↔ tela
- ✗ A cada mudança, lembrar de atualizar o DOM manualmente
- ✗ Código cresce rápido e fica difícil de manter

Com Vue.js

- ✓ Você declara o estado (data) — o Vue atualiza a tela sozinho
- ✓ Menos código repetitivo para sincronizar DOM
- ✓ Interface reage automaticamente quando os dados mudam
- ✓ Estrutura clara para crescer o projeto

Ambos resolvem o mesmo problema — mas o Vue tira de você o trabalho manual de sincronização.

O que é o Vue.js?

Um framework JavaScript para construir interfaces. O coração do Vue é a reatividade: você muda um dado, e o Vue atualiza a tela sozinho — sem você tocar no DOM.



O ciclo se repete a cada mudança de estado — é a reatividade acontecendo.

Instalando via CDN

Sem Node.js, sem npm. Só um `<script>` no HTML — igual às bibliotecas que vocês já usaram.

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Minha primeira app Vue</title>
</head>
<body>
  <div id="app">
    {{ mensagem }}
  </div>

  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

Repare

- O `<div id="app">` é onde o Vue vai atuar
- `{{ mensagem }}` já é sintaxe do Vue (interpolação)
- O script do Vue vem antes do nosso `app.js`

createApp() e mount()

Toda aplicação Vue começa com essas duas chamadas, no arquivo app.js:

```
// app.js
const { createApp } = Vue;

createApp({
  data() {
    return {
      mensagem: "Olá, turma!"
    };
  }
}).mount("#app");
```

createApp({ ... })

Cria a aplicação Vue a partir de um objeto de configuração

data()

Função que retorna os dados reativos da aplicação

.mount("#app")

Conecta a aplicação Vue ao elemento do HTML com esse id

Resultado: o texto "Olá, turma!" aparece dentro da <div id="app">.

data() e interpolação {{ }}

Tudo que está dentro de data() pode ser exibido no HTML com chaves duplas.

```
// app.js
createApp({
  data() {
    return {
      nome: "Maria",
      idade: 22
    };
  }
}).mount("#app");
```

```
<!-- index.html -->
<div id="app">
  <p>Nome: {{ nome }}</p>
  <p>Idade: {{ idade }}</p>
  <p>{{ nome + " tem " + idade + " anos" }}</p>
</div>
```

{{ }} aceita qualquer expressão JavaScript simples — variáveis, contas, concatenação de texto. Não é possível colocar blocos de código (if, for) direto ali dentro.

v-bind: atributos dinâmicos

`{{ }}` funciona só dentro do texto. Para atributos de tags HTML, usamos `v-bind`.

```
// app.js
createApp({
  data() {
    return {
      urlImagem: "logo-vue.png",
      linkSite: "https://vuejs.org"
    };
  }
}).mount("#app");
```

```
<!-- index.html -->


<!-- forma abreviada (mais usada) -->


<a :href="linkSite">Site do Vue</a>
```

`v-bind:atributo="dado"` pode ser escrito como `:atributo="dado"` — os dois fazem a mesma coisa.

v-if, v-else e v-show

Três diretivas para mostrar ou esconder elementos conforme uma condição.

```
<div id="app">
  <p v-if="logado">Bem-vindo(a) de volta!</p>
  <p v-else>Faça login para continuar</p>

  <p v-show="carregando">Carregando...</p>
</div>
```

v-if

Adiciona ou remove o elemento do HTML

v-else

Complementa um v-if anterior

v-show

Sempre no HTML; alterna CSS display

v-if x v-show: quando usar cada um

Critério	<code>v-if</code>	<code>v-show</code>
Elemento no HTML	Só existe se a condição for verdadeira	Sempre existe (fica com display:none)
Custo de alternância	Mais alto (cria/destrói elemento)	Mais baixo (só troca CSS)
Melhor uso	Condição muda pouco (ex: login)	Condição muda muito (ex: abas, loading)

Regra prática: começando agora, use v-if na maioria dos casos — troque para v-show só quando perceber alternância muito frequente.

Contador de cliques + mensagem

O que construir

- Um botão que incrementa um número na tela a cada clique (v-on:click / @click)
- O número inicial deve vir de data()
- Uma mensagem que aparece só quando o contador passar de 5, usando v-if
- Um botão “Zerar” que volta o contador para 0

Dica

```
data() {  
  return {  
    contador: 0  
  };  
},  
methods: {  
  incrementar() {  
    this.contador++;  
  }  
}
```

Recapitulando

CDN

Vue pode ser usado direto via `<script>`, sem build tools

`createApp` / `mount`

Toda aplicação começa criando e montando a instância

`data()`

Onde vivem os dados reativos da aplicação

`{{ }}` e `v-bind`

Exibem dados no texto e nos atributos do HTML

`v-if` / `v-show`

Controlam a exibição condicional de elementos

Próxima aula: Reatividade, formulários e listas (`v-model`, `v-for`, `computed properties`)